

RESUME ANALYZER

TECHNICAL DOCUMENTATION

AI-Powered Resume Analysis System using RAG Pipeline, FAISS Vector Search, and Google Gemini API

v1.0.0

Production-Ready

May 3, 2026

Engineering Team · HireLens Project

Table of Contents

- 1** Project Overview & Goals
 - Purpose & Core Features
 - Design Decisions & Trade-offs
- 2** System Architecture
 - High-Level Data Flow
 - Component Responsibilities
 - Request/Response Flows
- 3** Folder Structure & Module Explanation
 - Directory Layout
 - Key Module Explanations (Frontend & Backend)
- 4** API Reference
 - Authentication Endpoints
 - Resume Analysis Endpoints
- 5** Authentication & Authorization Flow
 - Token Lifecycle
 - JWT Structure
 - Authorization Model
 - CORS Configuration
- 6** Data Model & Storage
 - User Data Model
 - Resume Session Model
 - Analysis Result Schema
 - Entity Relationship
- 7** Deployment & Operations
 - Local Development Setup
 - Production Deployment (Render + Vercel)
 - Rollback Strategy
 - Monitoring & Debugging
- 8** Known Limitations & Gotchas
 - Architectural Limitations
 - CORS Issues

- Auth Gotchas
- PDF & Embedding Issues
- Rate Limiting & Security

A Appendix

- Environment Variable Template
- Useful Commands
- References

1 Project Overview & Goals

Resume Analyzer is an AI-powered Retrieval-Augmented Generation (RAG) system that evaluates resumes against job descriptions. It combines local on-device embeddings, FAISS vector search, and Google Gemini API to produce structured, actionable analysis reports for candidates and recruiters.

Core Features

Feature	Implementation	Trade-off
Resume Upload	Multipart form-data, max 5MB	Keeps request light; no chunked uploads
Local Embeddings	Xenova (on-device, all-MiniLM-L6-v2)	Free & private; slower cold start (~15-30s)
Vector Search	FAISS in-memory + disk persistence	Fast retrieval (~100ms); limited horizontal scale
Structured Analysis	Gemini API JSON mode	Reliable parsing; external API dependency
Session Tracking	UUID per upload → FAISS index	Stateless resume requests; cleanup required
JWT Auth	7-day expiration, localStorage	Simple; not refresh-token-based

Design Decisions & Trade-offs

Local Embeddings over API Embeddings Performance	Cost-free, privacy-respecting, no rate limits. Cold start ~15-30s on first use; CPU-bound inference. Future: cache model in container or use GPU acceleration.
FAISS over Traditional Database Architecture	Sub-millisecond similarity search; no schema migration overhead. Ephemeral (lost on container restart); limited horizontal scaling. Workaround: persist to disk + cleanup cron.
Gemini API over Local LLMs Quality	Higher quality structured output; no local GPU required. Rate-limited (15 req/min free tier); external dependency; latency 1-3s. Future: OpenAI/LM Studio fallback.
Single-page Upload (no batch) UX	Simpler UX; matches typical interview workflow (one resume per job). Users cannot compare multiple resumes at once. Future: batch API endpoint.

2 System Architecture

High-Level Data Flow

The system follows a two-phase pipeline: (1) **Ingestion** — PDF is parsed, chunked, embedded, and stored in FAISS; (2) **Analysis** — job description is embedded, top-k resume chunks are retrieved, and Gemini produces structured output.

Phase	Step	Component	Output
Ingestion	1. Parse PDF	pdf-parse library	Raw text + page count
	2. Chunk Text	embedder.js	250-char overlapping chunks
	3. Embed Chunks	Xenova (all-MiniLM-L6-v2)	384-dim vectors[]
	4. Store Index	FAISS + disk	faiss.index + docstore.json
Analysis	5. Embed Job Description	Xenova	Query vector
	6. Similarity Search	FAISS top-k	Top 5 resume chunks
	7. Build Prompt	ragChain.js	Structured prompt with context
	8. Call Gemini	ai.service.js	Structured JSON response
	9. Return Result	Express route	ATS score, skills, suggestions

Component Responsibilities

Frontend (React/Vite)

- Auth: Sign up, login, logout via JWT tokens stored in localStorage
- Upload: Drag-drop PDF via multipart form submission
- Analysis: Input job description and receive structured results
- Results Display: Render ATS score, match %, skills gap, suggestions

Backend (Express.js)

- Auth Middleware: Validate JWT tokens from Authorization header
- PDF Handler: Accept multipart upload, extract text via pdf-parse

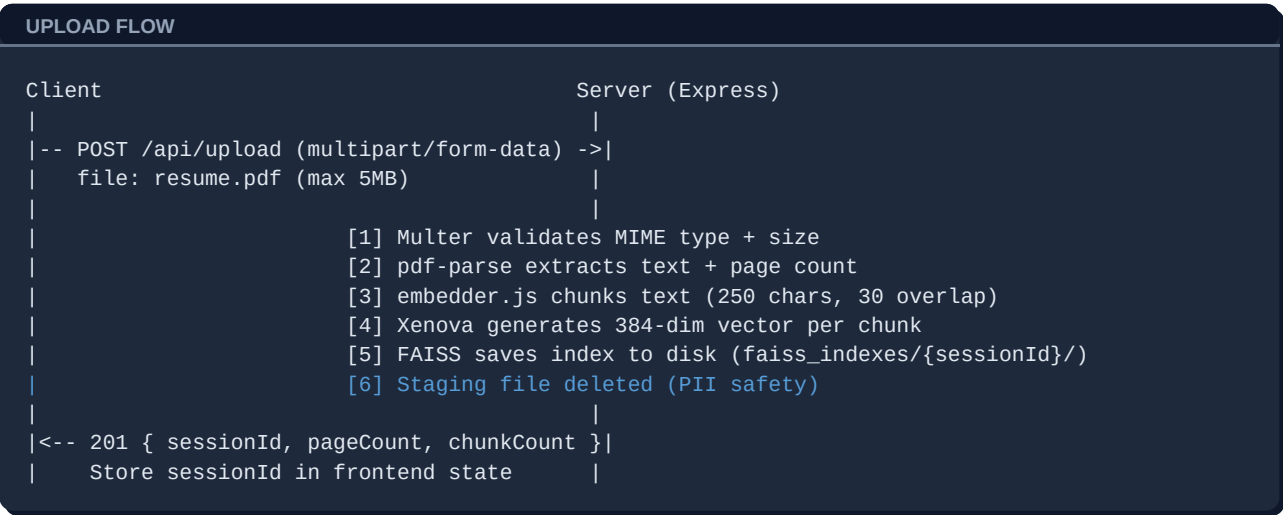
- Embedder: Generate 384-dim vectors using Xenova (on-device)
- Vector Store (FAISS): Create, save, load, search indices
- RAG Chain: Orchestrate prompt building, Gemini call, JSON parsing
- Cleanup: Hourly cron job to delete expired FAISS indices

External Dependencies

- Gemini API: Structured JSON analysis (15 req/min on free tier)
- Xenova Transformers: On-device embeddings (CPU-bound, no API cost)
- FAISS: In-memory vector similarity search

Request / Response Flows

Resume Upload Flow (POST /api/upload)



Analysis Flow (POST /api/analyze)

ANALYSIS FLOW

```
Client                                Server (Express)
|                                    |
|-- POST /api/analyze (JSON) -----> |
|   { sessionId, jobDescription }      |
|                                    |
|                                    |
|          [1] Load FAISS index for sessionId from disk
|          [2] Embed job description -> query vector
|          [3] FAISS similarity search -> top-5 resume chunks
|          [4] Build structured prompt (system + context + JD)
|          [5] Gemini API call with JSON mode enforced
|          [6] Parse + validate JSON response fields
|          [7] Return typed result object
|                                    |
|<-- 200 { ats_score, match_percentage, |
|         missing_skills, present_skills, |
|         suggestions, meta }            |
```

3 Folder Structure & Module Explanation

Directory Layout

PROJECT STRUCTURE

```
resume-analyzer/
├── client/                                # React/Vite frontend
│   ├── src/
│   │   ├── App.jsx                      # Main component – multi-state manager
│   │   ├── main.jsx                    # React entry point
│   │   ├── index.css                  # Tailwind CSS global styles
│   │   ├── components/
│   │   │   ├── DropZone.jsx           # PDF upload drag-drop zone
│   │   │   ├── AnalyzingState.jsx     # Progress indicator during analysis
│   │   │   ├── ResultsPanel.jsx       # Display ATS score, skills, suggestions
│   │   │   ├── EmptyState.jsx         # Idle state (no upload yet)
│   │   │   └── ScoreRing.jsx          # Circular progress ring for ATS %
│   │   ├── hooks/
│   │   │   └── useAnalyzer.js         # Multi-step state machine
│   │   ├── context/
│   │   │   └── AuthContext.jsx        # Global auth state + JWT management
│   │   └── utils/
│   │       ├── api.js                 # Axios instance + interceptors
│   │       └── vite.config.js         # Vite build config (port 3000)
│   └── server/                         # Express.js backend
│       ├── index.js                   # App entry – middleware + route binding
│       ├── routes/
│       │   ├── auth.js                # POST /register, /login, GET /me
│       │   ├── analyze.js             # POST /upload, POST /analyze
│       │   └── middleware/
│       │       ├── auth.js            # JWT verification middleware
│       │       ├── errorHandler.js    # Global error handler (4-arg signature)
│       │       ├── upload.js          # Multer config – PDF validation + size limit
│       │       └── sanitize.js        # Input validation for /analyze
│       ├── services/
│       │   ├── pdfExtractor.js        # Extract text from PDF file path
│       │   ├── embedder.js            # Chunk text + generate embeddings
│       │   ├── vectorStore.js         # FAISS: save/load/search indices
│       │   ├── ragChain.js            # Orchestrate RAG pipeline + Gemini
│       │   ├── ai.service.js          # Gemini API wrapper (JSON mode)
│       │   └── user.service.js        # User CRUD (bcrypt + JSON storage)
│       ├── utils/
│       │   ├── logger.js              # Winston/custom structured logger
│       │   ├── cleanup.js             # Cron: delete expired FAISS indices
│       │   └── validateEnv.js         # Ensure required env vars at startup
│       ├── data/
│       │   ├── users.json             # JSON file user storage
│       │   ├── faiss_indexes/         # Dynamic FAISS index storage
│       │   │   ├── {sessionId}/
│       │   │   │   ├── faiss.index    # Binary FAISS index
│       │   │   │   └── docstore.json  # Metadata + chunk content
```

Key Module Explanations

AuthContext.jsx

client/src/context/

Global auth state management via React Context API.

- State: user { id, email }, token (JWT in localStorage), loading flag
- Methods: register(email, password), login(email, password), logout()
- On token change: injects Authorization header into axios defaults
- On mount: validates existing token via GET /auth/me
- Gotcha: Token is not auto-refreshed. Expiry forces re-login.

useAnalyzer.js

client/src/hooks/

Multi-step state machine for the upload-analyze-results flow.

- States: READY → UPLOADING → ANALYZING → DONE (or ERROR)
- handleFile(): triggers POST /upload, transitions to ANALYZING on success
- handleAnalyze(): triggers POST /analyze, transitions to DONE on success
- handleReset(): returns to READY state, clears sessionId
- Gotcha: State is in-memory only — lost on page reload.

pdfExtractor.js

server/services/

Extract text from uploaded PDF files using pdf-parse.

- Reads file from Multer staging path, parses via PDFParse
- Returns { text, pageCount, metadata }
- File is deleted immediately after extraction (PII safety)
- Breaking change v2.4.5: Use named export { PDFParse } not default
- Limitation: Text-based PDFs only; scanned images return 422

embedder.js

server/services/

Convert text chunks into 384-dimensional vector representations using Xenova.

- chunkText(text, sessionId): splits into 250-char chunks with 30-char overlap
- Each chunk embedded to 384-dim float vector via all-MiniLM-L6-v2
- Returns Document[] with pageContent + metadata (sessionId, chunkIndex)
- Performance: First call ~15-30s (model download); subsequent ~100-500ms
- Gotcha: Synchronous embedding blocks Node.js event loop under high load

vectorStore.js

server/services/

FAISS index lifecycle — create, save, load, search, and delete.

- saveIndex(): creates FAISS index, writes faiss.index + docstore.json
- loadIndex(): reads binary index + docstore from disk into memory
- similaritySearch(): embeds query, finds top-k nearest chunks by cosine similarity
- deleteIndex(): removes sessionId directory (called by cleanup cron)
- Limitation: Single-process FAISS cannot scale horizontally

ragChain.js

server/services/

Orchestrates the full RAG pipeline from retrieval to LLM response.

- Loads FAISS index, embeds job description, retrieves top-5 chunks
- Builds structured prompt: system role + schema + context + job description
- Calls Gemini API with json_object response format enforced
- Validates all required fields: ats_score, match_percentage, missing_skills, present_skills, suggestions
- On invalid JSON: retries with more explicit prompt instruction

4 API Reference

Authentication Endpoints

POST

/api/auth/register

Register a new user account

Request Body:

JSON

```
{
  "email": "user@example.com",
  "password": "securePassword123"
}
```

Response (Success):

JSON

```
{
  "user": { "id": "1777805104195", "email": "user@example.com" },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "message": "User registered successfully"
}
```

Error Codes:

HTTP Code	Meaning
400	Email already registered, or password < 6 characters
500	Server error (bcrypt failure, file I/O)

Password is hashed with bcryptjs (salt rounds = 10) before storage.

Email must be unique across all users.

POST

/api/auth/login

Authenticate with email and password

Request Body:

JSON

```
{
  "email": "user@example.com",
  "password": "securePassword123"
}
```

Response (Success):

JSON

```
{
  "user": { "id": "1777805104195", "email": "user@example.com" },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "message": "Logged in successfully"
}
```

Error Codes:

HTTP Code	Meaning
401	Email not found, or password mismatch
500	Server error

GET	/api/auth/me	Retrieve current user profile (requires JWT)
-----	--------------	--

Response (Success):

JSON

```
{
  "user": { "id": "1777805104195", "email": "user@example.com" }
}
```

Error Codes:

HTTP Code	Meaning
401	Missing Authorization header, or token invalid/expired
404	User not found

Resume Analysis Endpoints

POST	/api/upload	Upload a resume PDF and create FAISS index
------	-------------	--

Request Body:

JSON

```
Content-Type: multipart/form-data
field "file": PDF file (max 5MB)
```

Response (Success):

```
JSON

{
  "sessionId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "pageCount": 2,
  "chunkCount": 8,
  "message": "Resume uploaded and indexed successfully"
}
```

Error Codes:

HTTP Code	Meaning
400	No file in request
401	Missing/invalid JWT
413	File exceeds 5MB limit
415	Not a PDF file
422	PDF text extraction failed (corrupted or scanned image)
500	Embedding generation failed

Store the returned sessionId in frontend state — required for POST /analyze.

Session expires after 24 hours (INDEX_TTL_HOURS). Re-upload required after expiry.

■ Rate Limit

Default: 10 uploads per 15 minutes per user. Configurable via RATE_LIMIT_MAX_UPLOAD.

POST	/api/analyze	Analyze resume against a job description via RAG + Gemini
------	--------------	---

Request Body:

```
JSON

{
  "sessionId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "jobDescription": "Senior React Developer with 5+ years..."
}
```

Response (Success):

JSON

```
{
  "ats_score": 78,
  "match_percentage": 76,
  "missing_skills": ["GraphQL", "AWS DevOps", "Kubernetes"],
  "present_skills": ["React", "TypeScript", "Jest", "Next.js"],
  "suggestions": [
    "Add quantified impact metrics (e.g., improved performance by 40%)",
    "Highlight leadership experience for senior role"
  ],
  "meta": {
    "sessionId": "a1b2c3d4-...",
    "processingTimeMs": 3247,
    "model": "gemini-2.5-flash"
  }
}
```

Error Codes:

HTTP Code	Meaning
400	Missing or invalid input fields
401	Missing/invalid JWT
404	Session / FAISS index not found or expired
429	Gemini API rate limit exceeded (15 req/min free tier)
500	Gemini API down, or JSON parsing failed

Average response time: 2-4 seconds (includes Gemini API latency).

Job description must be at least 20 characters.

■ Rate Limit

Default: 20 analyses per 15 minutes per user. Configurable via RATE_LIMIT_MAX_ANALYZE.

5 Authentication & Authorization Flow

JWT Token Structure

Component	Value / Description
Algorithm	HS256 (HMAC-SHA256)
Payload: userId	Unique user identifier (timestamp-based)
Payload: iat	Issued-at timestamp (Unix epoch)
Payload: exp	Expiration: iat + 7 days (604,800 seconds)
Signature	HMAC-SHA256(header.payload, JWT_SECRET)
Storage	localStorage["token"] on client; axios Authorization header

Authorization Permission Matrix

Endpoint	Auth Required	Scope
POST /auth/register	No	Public — anyone can sign up
POST /auth/login	No	Public
GET /auth/me	Yes (JWT)	User-scoped — own profile only
POST /upload	Yes (JWT)	User-scoped — own uploads only
POST /analyze	Yes (JWT)	User-scoped — own sessions only

CORS Configuration

Current configuration echoes the requesting origin to support both local development (localhost:3000) and production (Vercel frontend → Render backend). Credentials mode is enabled for authenticated requests.

CORS CONFIGURATION

```
// Current (permissive – development/MVP)
app.use((req, res, next) => {
  const origin = req.get('origin') || '*'
  res.header('Access-Control-Allow-Origin', origin)
  res.header('Access-Control-Allow-Methods', 'GET, POST, DELETE, OPTIONS')
  res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization')
  res.header('Access-Control-Allow-Credentials', 'true')
  if (req.method === 'OPTIONS') res.sendStatus(200)
  else next()
})

// Recommended (production hardening)
const allowedOrigins = [
  'https://resume-analyzer-beta-rose.vercel.app',
  'http://localhost:3000'
]
app.use(cors({
  origin: (origin, cb) => cb(null, allowedOrigins.includes(origin)),
  credentials: true
}))
```


6 Data Model & Storage

User Data Model

USER SCHEMA

```
// server/data/users.json
[
  {
    "id": "1777805104195",           // timestamp-based unique ID
    "email": "user@example.com",     // unique constraint enforced in-memory
    "password": "$2a$10$...",       // bcryptjs hash (salt rounds = 10)
    "createdAt": "2026-05-03T11:00:00Z"
  }
]
```

■ Scaling Limitation

JSON file storage is suitable for MVP only. Migrate to PostgreSQL or MongoDB for production scale.

Data is loaded entirely in-memory on server start — not suitable for >10k users.

Resume Session Data Model (FAISS)

DOCSTORE SCHEMA

```
// faiss_indexes/{sessionId}/docstore.json
{
  "sessionId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "fileName": "resume.pdf",
  "uploadedAt": "2026-05-03T11:00:00Z",
  "expiresAt": "2026-05-04T11:00:00Z", // uploadedAt + INDEX_TTL_HOURS
  "chunks": [
    {
      "index": 0,
      "pageContent": "Senior Software Engineer with 5 years...",
      "metadata": { "chunkIndex": 0, "sessionId": "..." }
    }
  ]
  // ... up to ~200 chunks for a 50-page resume
}
```

File	Format	Size (typical)	Purpose
faiss.index	Binary FAISS	50–500 KB	Vector index for similarity search
docstore.json	JSON	10–200 KB	Chunk text + metadata

Analysis Result Schema (In-Memory)

ANALYSIS RESULT SCHEMA

```
// Returned by POST /api/analyze – not persisted to DB in v1.0
{
  "ats_score": 78,                // 0-100 integer
  "match_percentage": 76,        // 0-100 integer
  "missing_skills": ["GraphQL", "AWS DevOps"], // max 10 items
  "present_skills": ["React", "TypeScript"], // max 10 items
  "suggestions": ["Add quantified metrics"], // max 5 items
  "meta": {
    "sessionId": "a1b2c3d4-...",
    "processingTimeMs": 3247,
    "model": "gemini-2.5-flash"
  }
}
```

Note: Analysis results are not persisted in v1.0.0. Future versions will store results in a database to support history and comparison features.

7 Deployment & Operations

Local Development Setup

Prerequisites: Node.js 20.x, npm or yarn

LOCAL DEV SETUP

```
# Backend (Express.js - port 5001)
cd server
npm install
cp .env.example .env
# Fill in: GEMINI_API_KEY, JWT_SECRET
npm run dev

# Frontend (React/Vite - port 3000)
cd client
npm install
npm run dev
```

Environment Variables Reference

Variable	Default	Description
PORT	5001	Express server port
NODE_ENV	development	Set to "production" on Render
JWT_SECRET	(required)	Secret for signing JWT tokens — use 32+ random chars
GEMINI_API_KEY	(required)	Google Gemini API key from aistudio.google.com
GEMINI_MODEL	gemini-2.5-flash	Model identifier
MAX_FILE_SIZE_MB	5	Resume file size limit in megabytes
RATE_LIMIT_WINDOW_MS	900000	Rate limit window (15 minutes)
RATE_LIMIT_MAX_UPLOAD	10	Max uploads per window per user
RATE_LIMIT_MAX_ANALYZE	20	Max analyses per window per user
INDEX_TTL_HOURS	24	FAISS index expiration in hours
FAISS_INDEX_DIR	./faiss_indexes	Index storage path on disk
UPLOAD_DIR	./uploads	Temporary PDF staging directory
HUGGINGFACE_EMBEDDING_MODEL	Xenova/all-MiniLM-L6-v2	Embedding model identifier

Production Deployment — Render + Vercel

Layer	Platform	Trigger	URL
Backend (API)	Render (free tier)	Push to main branch	hirelens-api-node.onrender.com
Frontend (React)	Vercel	Push to main branch	resume-analyzer-beta-rose.vercel.app

Rollback Strategy:

- **Backend (Render):** Render dashboard → Service → Deploys → select previous → Redeploy
- **Frontend (Vercel):** Vercel dashboard → Project → Deployments → "." → Promote to Production
- **Git fallback:** git tag stable releases. git checkout v1.0.0 && git push origin main --force (coordinate with team first)

Monitoring & Health Checks

MONITORING

```
# View Render backend logs
curl -H "Authorization: Bearer $RENDER_API_KEY" \
  https://api.render.com/v1/services/srv-{id}/events

# Health endpoint (future implementation)
GET /health
--> { "status": "ok", "uptime": 123456, "version": "1.0.0" }

# Frontend debugging
# DevTools → Console: check for CORS errors, 401 auth failures, 404 endpoints
# DevTools → Network: inspect full request/response pairs
```

8 Known Limitations & Gotchas

Architectural Limitations

Limitation	Impact	Recommended Fix
Single-server FAISS	Cannot scale horizontally; all vectors in one process	Migrate to Pinecone, Weaviate, or Milvus
In-memory embeddings	CPU-bound; blocks event loop under high concurrency	Offload to worker threads or dedicated GPU server
Gemini API dependency	Outage causes complete app failure	Implement OpenAI/LM Studio fallback
No refresh tokens	Users must re-authenticate every 7 days	Implement refresh token rotation
JSON file user storage	Not suitable for production scale (>1k users)	Migrate to PostgreSQL or MongoDB
No index sharding	FAISS indices must fit entirely in RAM	Implement multi-shard architecture
No result persistence	Analysis history lost on page reload	Store results in DB; add history endpoint

Known Issues & Resolved Bugs

■ CORS Issues — Resolved May 3, 2026

Root cause: Backend not setting Access-Control-Allow-Origin correctly for Vercel → Render requests.

Fix: Added explicit CORS middleware echoing requesting origin + Access-Control-Allow-Credentials: true.

Future hardening: Restrict CORS to specific allowed origins list.

■ Auth Token Not Injected into API Requests

Cause: api.js creates a separate axios instance; does not inherit axios.defaults.headers.

Fix: Added request interceptor to inject token from localStorage on every request.

Lesson: Always centralize auth header injection via interceptors, never manual headers.

■ Scanned PDFs Fail Extraction (422)

Cause: pdf-parse extracts text-layer only; scanned images have no text layer.

Impact: Users uploading scanned/image PDFs receive 422 Unprocessable Entity.

Future: Integrate Tesseract OCR or Google Vision API for image-based PDFs.

■ Xenova First-Call Slowdown (~15-30s)

Cause: Model downloads from HuggingFace on first inference (not cached).

Impact: First upload on cold server appears hung; subsequent uploads fast.

Mitigation: Pre-warm embeddings on server startup or use containerized cached model.

■ Render Backend Redeploy Lag (2-5 minutes)

Cause: Git push triggers Render rebuild + restart pipeline.

Impact: Frontend may run new code while backend runs old code — CORS or API mismatch.

Mitigation: Deploy backend first, verify health, then deploy frontend.

Security Posture Summary

Control	Status	Detail
Password hashing	Implemented	bcryptjs, salt rounds = 10
JWT authentication	Implemented	HS256, 7-day expiry
CORS configuration	Implemented	Permissive (harden for production)
File size limits	Implemented	5MB max, PDF MIME check
HTTPS enforcement	Platform-managed	Handled by Vercel and Render
Rate limiting (auth)	Missing	No brute-force protection on /login, /register
Input sanitization	Missing	Job description field vulnerable to prompt injection
Audit logging	Missing	No API call audit trail
Refresh tokens	Missing	Fixed 7-day JWT with no rotation

A Appendix

Environment Variable Template

.ENV TEMPLATE

[illegible]

PORT=5001

NODE_ENV=development

```
JWT_SECRET=your-super-secret-jwt-key-change-in-production-min32chars
```

[illegible]

GEMINI_API_KEY=AIzaSy...

```
GEMINI_MODEL=gemini-2.5-flash
```

[illegible]

```
MAX_FILE_SIZE_MB=5
```

```
UPLOAD_DIR=./uploads
```

[illegible]

```
RATE_LIMIT_WINDOW_MS=900000 # 15 minutes
```

RATE_LIMIT_MAX_UPLOAD=10

RATE_LIMIT_MAX_ANALYZE=20

```
# ■■ FAISS / Vector Store ██████████
```

```
FAISS_INDEX_DIR=./faiss_indexes
```

INDEX_TTL_HOURS=24

[illegible]

CHUNK SIZE=250

CHUNK_OVERLAP=30

TOP_K_RESULTS=5

[illegible]

HUGGINGFACE EMBEDDING MODEL=Xenova/all-MiniLM-L6-v2

[illegible]

```
VITE_API_URL=http://localhost:5001 # or production URL
```

Useful Commands

USEFUL COMMANDS

```
# ■■■ Backend
cd server
npm run dev          # Start dev server with auto-reload (nodemon)
npm start            # Start production server
node utils/cleanup.js # Manually delete expired FAISS indices
npm test             # Run test suite (if configured)

# ■■■ Frontend
cd client
npm run dev          # Start dev server on port 3000
npm run build        # Build for production (output to dist/)
npm run preview      # Preview production build locally

# ■■■ Git Workflow
git tag v1.0.0       # Tag stable release
git push origin --tags # Push tags to remote
```

References & Documentation

Resource	URL
LangChain.js	https://js.langchain.com/
FAISS GitHub	https://github.com/facebookresearch/faiss
Xenova Transformers.js	https://xenova.github.io/transformers.js/
Google Gemini API	https://ai.google.dev/docs
JWT Specification	https://jwt.io/
Express.js Guide	https://expressjs.com/
React Documentation	https://react.dev/
Vite Documentation	https://vitejs.dev/